

Project 0: Virtual Machine

Cloud Computing (Spring 2026)

Student Name: **Adewole Adeoshun**

Cuny ID: 24081306

Student Type: Undergraduate Student

Professor: Jun Li

Tasks Completed: 1-7

Due Date: February 17, 2026

Amazon Web Services (AWS)

Region: us-east-1 (US East - N. Virginia)

TASK 1: Environment Setup

Commands Run:

Step 1: Get my public IP address

curl -4 ifconfig.me

Output: 73.255.237.70

Step 2: Install AWS CLI (verification)

aws --version

Output: aws-cli/2.x.x Python/3.x.x Darwin/xx.x.x

Step 3: Configure AWS CLI

aws configure

Entered:

- AWS Access Key ID: [REDACTED]
- AWS Secret Access Key: [REDACTED]
- Default region: us-east-1
- Default output format: json

Step 4: Verify CLI configuration

aws sts get-caller-identity

Output:

```
{
  "UserId": "AIDA2MJRZ7EBCDGZCVNRI",
  "Account": "713605904642",
  "Arn": "arn:aws:iam::713605904642:user/project-user"
```

}

Screenshots for Task 1:

- Root account MFA enabled

The screenshot shows the AWS IAM console for the root account. The browser address bar indicates the URL: `us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/security_credentials`. The page title is "Security credentials".

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access Management

- User groups
- Users
- Roles
- Policies
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

New

▼ Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Credential report
- Organization activity

MFA device assigned

You can register up to 8 MFA devices of any combination of the currently supported MFA types with your AWS account root and IAM user. With multiple MFA devices, you only need one MFA device to sign in to the AWS console or create a session through the AWS CLI with that user.

Account details

Account name: Cloud Computing Project

Account ID: 713605904642

Email address: mycloudcomputing10@outlook.com

Canonical user ID: f077e486032dd1d9a7ff720ed256d57182b970be1958b21a6716bae1ea3064be

Multi-factor authentication (MFA) (1)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

Type	Identifier	Certifications	Created on
☐ Virtual	arn:aws:iam::713605904642:mfa/root-mfa	Not Applicable	Sat Feb 14 2026

Access keys (0)

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

Access key ID	Created on	Access key last used	Region last used	Service last used	Status
---------------	------------	----------------------	------------------	-------------------	--------

- IAM user project0-user with 5 permission policies

aws

Search

[Option+S]

Cloud Computing Project (7136-0590-4642)

Cloud Computing Project

IAM > Users > project-user

project-user info

Delete

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management

Temporary delegation requests

New

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Organization activity

Summary

ARN
arn:aws:iam::713605904642:user/project-user

Console access
Enabled with MFA

Access key 1
AKIA2MJRZ7EBIRDNOSWG - Active
Used today. Created yesterday.

Created
February 14, 2026, 19:31 (UTC-05:00)

Last console sign-in
Never

Access key 2
Create access key

Permissions

Groups

Tags (1)

Security credentials

Last Accessed

Permissions policies (5)

Remove

Add permissions

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type

Search

All types

1

Policy name

Type

Attached via

AmazonEC2FullAccess

AWS managed

Directly

AmazonVPCFullAccess

AWS managed

Directly

AutoScalingFullAccess

AWS managed

Directly

ElasticLoadBalancingFullAccess

AWS managed

Directly

IAMUserChangePassword

AWS managed

Directly

- aws sts get-caller-identity verification

```
[dewole@Dewoles-MacBook-Pro ~ % aws sts get-caller-identity
{
  "UserId": "AIDA2MJRZ7EBCDGZCVNRI",
  "Account": "713605904642",
  "Arn": "arn:aws:iam::713605904642:user/project-user"
}
dewole@Dewoles-MacBook-Pro ~ % █
```

Explanation:

I enabled Multi-Factor Authentication (MFA) on the root account to enhance security, following AWS best practices that the root account should only be used for initial setup. I created an IAM user named (project-user) changed to (project0-user) with specific permissions for EC2, ELB, VPC, and Auto Scaling services, following the principle of least privilege, this is granting only the permissions necessary for this project. I configured the AWS CLI with programmatic access credentials to allow command-line management of AWS resources.

The `aws sts get-caller-identity` command verified that my CLI is properly authenticated and operating under the IAM user (not root), which is the correct security posture for daily operations.

TASK 2: Single Instance Deployment

Commands Run:

Step 1: Get my public IP address

```
curl -4 ifconfig.me
```

Output: 73.255.237.70

Step 2: Create Security Group

```
aws ec2 create-security-group \  
--group-name project0-web-sg \  
--description "Security group for Project 0 web server" \  
--output json
```

Output: {
 "GroupId": "sg-076bb4041aa47160d"
}

Step 3: Add SSH rule (restricted to my IP only)

```
aws ec2 authorize-security-group-ingress \  
--group-id sg-076bb4041aa47160d \  
--protocol tcp \  
--port 22 \  
--cidr 73.255.237.70/32
```

Step 4: Add HTTP rule (open to public)

```
aws ec2 authorize-security-group-ingress \  
--group-id sg-076bb4041aa47160d \  
--protocol tcp \  
--port 80 \  
--cidr 0.0.0.0/0
```

Step 5: Verify security group configuration

```
aws ec2 describe-security-groups \  
--group-ids sg-076bb4041aa47160d \  
--query 'SecurityGroups[0].[GroupId,GroupName,Description]' \  
--output json
```

Output:

```
[  
  "sg-076bb4041aa47160d",  
  "project0-web-sg",  
  "Security group for Project 0 web server"
```

```
]
```

```
aws ec2 describe-security-groups \  
--group-id sg-076bb4041aa47160d \  
--query 'SecurityGroups[0].IpPermissions' \  
--output json
```

Output:[

```
{  
  "IpProtocol": "tcp",  
  "FromPort": 80,  
  "ToPort": 80,  
  "UserIdGroupPairs": [],  
  "IpRanges": [  
    {  
      "CidrIp": "0.0.0.0/0"  
    },  
  ],  
  "Ipv6Ranges": [],  
  "PrefixListIds": []  
},  
{  
  "IpProtocol": "tcp",  
  "FromPort": 22,  
  "ToPort": 22,  
  "UserIdGroupPairs": [],  
  "IpRanges": [  
    {  
      "CidrIp": "73.255.237.70/32"  
    },  
  ],  
  "Ipv6Ranges": [],  
  "PrefixListIds": []  
}  
]
```

Step 6: Create SSH key pair

```
aws ec2 create-key-pair \  
--key-name project0-key \  
--query 'KeyMaterial' \  

```

```
--output text > project0-key.pem
```

```
chmod 400 project0-key.pem
```

```
ls -la project0-key.pem
```

Output:

```
-r----- 1 dewole staff 1675 Feb 15 00:59 project0-key.pem
```

Step 7: Find Amazon Linux 2023 AMI

```
aws ec2 describe-images \  
--image-ids ami-0c1fe732b5494dc14 \  
--query 'Images[0].[ImageId,Name]' \  
--output text
```

Output:

```
ami-0c1fe732b5494dc14 al2023-ami-2023.10.20260202.2-kernel-6.1-x86_64
```

Step 8: Launch EC2 instance

```
aws ec2 run-instances \  
--image-id ami-0c1fe732b5494dc14 \  
--instance-type t3.micro \  
--key-name project0-key \  
--security-group-ids sg-076bb4041aa47160d \  
--tag-specifications  
'ResourceType=instance,Tags=[{Key=Name,Value=project0-instance-1}]' --output json
```

Output:

```
{  
  "Reservations": [  
    {  
      "ReservationId": "r-0026655d89ef3c0de",  
      "OwnerId": "713605904642",  
      "Instances": [  
        {  
          "InstanceId": "i-0b43e1f39c633c356",  
          "InstanceType": "t3.micro",  
          "ImageId": "ami-0c1fe732b5494dc14",  
          "KeyName": "project0-key",  
          "State": {  
            "Name": "running"  
          },  
        },  
      ],  
    },  
  ],  
}
```

```

    "PrivateDnsName": "ip-172-31-21-30.ec2.internal",
    "PrivateIpAddress": "172.31.21.30",
    "PublicDnsName": "ec2-54-145-9-155.compute-1.amazonaws.com",
    "PublicIpAddress": "54.145.9.155",
    "SubnetId": "subnet-0a3a42582ce46e5ca",
    "VpcId": "vpc-0c73dd5cf82ea5035",
    "SecurityGroups": [
      {
        "GroupId": "sg-076bb4041aa47160d",
        "GroupName": "project0-web-sg"
      }
    ]
  }
]
}

```

Step 9: Verify instance is running

```

aws ec2 describe-instances \
  --instance-ids i-0b43e1f39c633c356 \
  --query
'Reservations[0].Instances[0].[InstanceId,InstanceType,State.Name,PublicIpAddress,PrivateIpAddress,SubnetId]' \
  --output table

```

Output:

```

-----
|   DescribeInstances   |
+-----+
| i-0b43e1f39c633c356   |
| t3.micro                |
| running                |
| 54.145.9.155           |
| 172.31.21.30           |
| subnet-0a3a42582ce46e5ca |
+-----+

```

Step 10: Connect via SSH

```
ssh -i project0-key.pem ec2-user@54.145.9.155
```

Step 11: Install Apache web server

```
sudo dnf update -y
sudo dnf install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl status httpd
```

Output:

```
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-02-15 06:20:03 UTC; 14s ago
     Docs: man:httpd.service(8)
  Main PID: 25777 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec:
0 B/sec"
    Tasks: 177 (limit: 1067)
   Memory: 13.3M
      CPU: 90ms
  CGroup: /system.slice/httpd.service
          └─25777 /usr/sbin/httpd -DFOREGROUND
            └─25799 /usr/sbin/httpd -DFOREGROUND
              └─25800 /usr/sbin/httpd -DFOREGROUND
                └─25809 /usr/sbin/httpd -DFOREGROUND
                  └─25829 /usr/sbin/httpd -DFOREGROUND
```

```
Feb 15 06:20:03 ip-172-31-21-30.ec2.internal systemd[1]: Starting httpd.service - The
Apache HTTP Server...
Feb 15 06:20:03 ip-172-31-21-30.ec2.internal httpd[25777]: Server configured, listening
on: port 80
Feb 15 06:20:03 ip-172-31-21-30.ec2.internal systemd[1]: Started httpd.service - The
Apache HTTP Server.
```

Step 12: Get hostname and private IP

```
HOSTNAME=$(hostname)
PRIVATE_IP=$(hostname -I | awk '{print $1}')
echo "Hostname: $HOSTNAME"
echo "Private IP: $PRIVATE_IP"
```

Output:

```
Hostname: ip-172-31-21-30.ec2.internal
Private IP: 172.31.21.30
```

Step 13: Create custom HTML page

```
sudo bash -c 'cat > /var/www/html/index.html << "EOF"
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Project 0 - Cloud Computing</title>
```

```
  <style>
```

```
    body {
```

```
      font-family: Arial, sans-serif;
```

```
      display: flex;
```

```
      justify-content: center;
```

```
      align-items: center;
```

```
      height: 100vh;
```

```
      margin: 0;
```

```
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
```

```
    }
```

```
    .container {
```

```
      background: white;
```

```
      padding: 40px;
```

```
      border-radius: 10px;
```

```
      box-shadow: 0 10px 40px rgba(0,0,0,0.2);
```

```
      text-align: center;
```

```
    }
```

```
    h1 { color: #333; margin-bottom: 20px; }
```

```
    .info {
```

```
      background: #f0f0f0;
```

```
      padding: 20px;
```

```
      border-radius: 5px;
```

```
      margin-top: 20px;
```

```
    }
```

```
    .label { font-weight: bold; color: #667eea; }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <div class="container">
```

```
    <h1>Hello World</h1>
```

```
    <p>Welcome to Cloud Computing Project!</p>
```

```
    <div class="info">
```

```
      <p><span class="label">Hostname:</span> ip-172-31-21-30.ec2.internal</p>
```

```
      <p><span class="label">Private IP:</span> 172.31.21.30</p>
```

```
    </div>
```

```
  </div>
```

```
</body>
```

</html>
EOF'

Screenshots for Task 2:

- Security group rules (SSH from 73.255.237.70/32, HTTP from 0.0.0.0/0)

aws

Search

[Option+S]

United States (N. Virginia)

Cloud Computing Project (7158-0350-4042)

EC2

Security Groups

sg-076bb4041aa47160d - project0-web-sg

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

New

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

sg-076bb4041aa47160d - project0-web-sg

Actions

Details

Security group name

project0-web-sg

Security group ID

sg-076bb4041aa47160d

Description

Security group for Project 0 web ser
ver

VPC ID

vpc-0c73dd5cf82ea5035

Owner

713605904642

Inbound rules count

2 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)

Manage tags

Edit inbound rules

Search

IP version	Type	Protocol	Port range	Source	Description
IPv4	SSH	TCP	22	73.255.237.70/32	-
IPv4	HTTP	TCP	80	0.0.0.0/0	-

- **EC2 instance i-0b43e1f39c633c356 running**

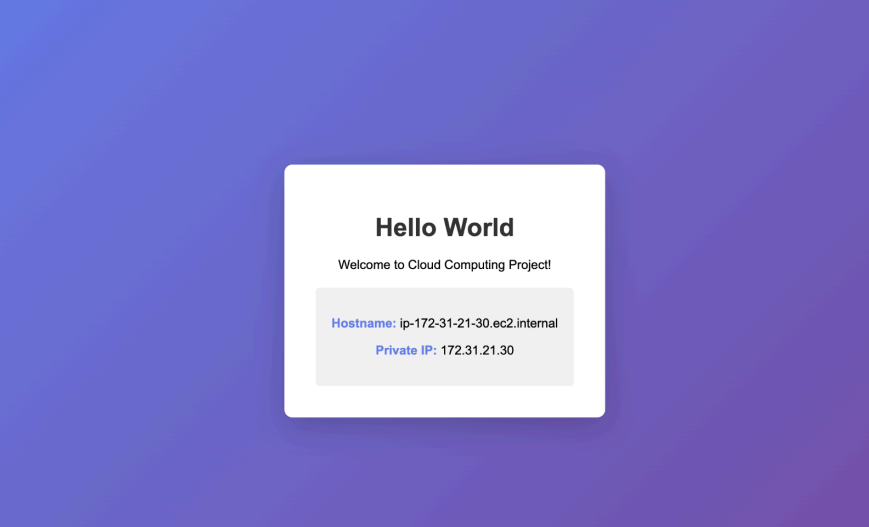
The screenshot displays the AWS Management Console interface for an EC2 instance. The top navigation bar shows the AWS logo, a search bar, and the current region (United States (N. Virginia)). The left sidebar contains navigation links for EC2, including Dashboard, EC2 Global View, Events, Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, and Capacity Manager. The main content area shows the 'Instance summary for i-0b43e1f39c633c356 (project0-instance-1)'. The instance is in the 'Running' state. Key details include: Public IPv4 address (54.145.9.155), Private IPv4 addresses (172.31.21.30), Public DNS (ec2-54-145-9-155.compute-1.amazonaws.com), Elastic IP addresses (none), AWS Compute Optimizer finding (Opt-in to AWS Compute Optimizer for recommendations), Auto Scaling Group name (none), and Managed (false).

Instance summary for i-0b43e1f39c633c356 (project0-instance-1)		
Updated less than a minute ago		
Instance ID i-0b43e1f39c633c356	Public IPv4 address 54.145.9.155 open address	Private IPv4 addresses 172.31.21.30
IPv6 address -	Instance state Running	Public DNS ec2-54-145-9-155.compute-1.amazonaws.com open address
Hostname type IP name: ip-172-31-21-30.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-21-30.ec2.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 54.145.9.155 [Public IP]	VPC ID vpc-0c73dd5cf82ea5035	Auto Scaling Group name -
IAM role -	Subnet ID subnet-0a3a42582ce46e5ca	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:713605904642:instance/i-0b43e1f39c633c356	
Operator -		

- **Successful SSH connection to ec2-user@ip-172-31-21-30**

```
#_
~\_####_ Amazon Linux 2023
~~\_#####\
~~\_###|
~~\_#/____ https://aws.amazon.com/linux/amazon-linux-2023
~~V~' '~>
~~~~
~~~~
~~._.-/_
~/./_/
~/m/'
```

Last login: Sun Feb 15 06:16:01 2026 from 73.255.237.70
[ec2-user@ip-172-31-21-30 ~]\$ █



The screenshot shows a web browser window with a purple background. In the center, there is a white rectangular box containing the following text:

Hello World

Welcome to Cloud Computing Project!

Hostname: ip-172-31-21-30.ec2.internal

Private IP: 172.31.21.30

The browser's address bar at the top shows the URL '54.145.9.155' and a 'Not Secure' warning. The browser's toolbar includes back, forward, and refresh buttons, as well as icons for bookmarks, extensions, and a menu.

Explanation:

I created a security group with two rules following the principle of least privilege: SSH access (port 22) restricted to only my IP address (73.255.237.70/32) to prevent unauthorized access, and HTTP access (port 80) open to the internet (0.0.0.0/0) since the web server needs to be publicly accessible. I launched a t3.micro EC2 instance using Amazon Linux 2023 to stay within the AWS free tier. After connecting via SSH using the generated key pair, I installed Apache HTTP server and configured it to start automatically on boot using systemctl enable. I created a custom index.html page that displays the server's hostname and private IP address, which is crucial for demonstrating load balancing in Phase 2 - when traffic is distributed across multiple servers, we can visually confirm which server is responding by seeing different hostnames/IPs.

TASK 3: Custom Machine Image (AMI)

Commands Run:

Step 1: Create AMI from running instance

```
aws ec2 create-image \  
  --instance-id i-0b43e1f39c633c356 \  
  --name "project0-web-ami" \  
  --description "Amazon Linux 2023 with Apache pre-installed for Project 0" \  
  --no-reboot
```

Output:

```
{  
  "ImageId": "ami-0d73205c3dd1af0e9"  
}
```

Step 2: Verify AMI is available

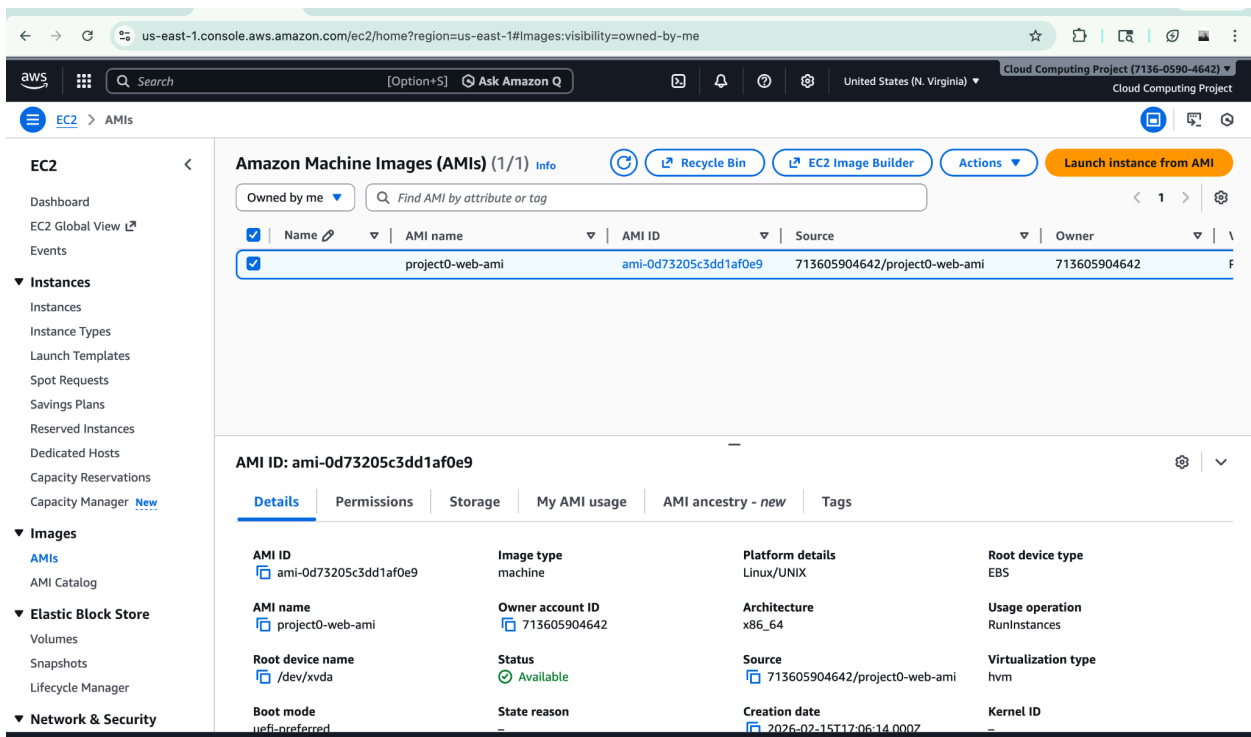
```
aws ec2 describe-images \  
  --image-ids ami-0d73205c3dd1af0e9 \  
  --query 'Images[0].[ImageId,State,Name,Description]' \  
  --output json
```

Output:

```
[
  "ami-0d73205c3dd1af0e9",
  "available",
  "project0-web-ami",
  "Amazon Linux 2023 with Apache pre-installed for Project 0"
]
```

Screenshots for Task 3:

- AMI ami-0d73205c3dd1af0e9 (project0-web-ami) available



Explanation:

Instead of manually installing Apache on every new server, I created a custom Amazon Machine Image (AMI) from the configured instance. This AMI captures the entire system state including the installed Apache web server, the custom HTML file, and all configurations. This image serves as a template for launching identical instances quickly and consistently in Phase 2, ensuring all servers have the same software configuration without requiring manual setup. The `--no-reboot` flag kept the instance running during the AMI creation process.

TASK 4: Load Balancing & Redundancy

Commands Run:

Step 1: Get VPC ID

```
aws ec2 describe-vpcs \
  --filters "Name=isDefault,Values=true" \
  --query 'Vpcs[0].VpcId' \
  --output text
```

Output:

```
vpc-0c73dd5cf82ea5035
```

Step 2: Create Target Group

```
aws elbv2 create-target-group \
  --name project0-tg \
  --protocol HTTP \
  --port 80 \
  --vpc-id vpc-0c73dd5cf82ea5035 \
  --health-check-enabled \
  --health-check-path / \
```

```
--health-check-interval-seconds 30 \  
--health-check-timeout-seconds 5 \  
--healthy-threshold-count 2 \  
--unhealthy-threshold-count 2 \  
--output json
```

Output:

```
{  
  "TargetGroups": [  
    {  
      "TargetGroupArn":  
"arn:aws:elasticloadbalancing:us-east-1:713605904642:targetgroup/project0-tg/c888dfa7  
51371cc7",  
      "TargetGroupName": "project0-tg",  
      "Protocol": "HTTP",  
      "Port": 80,  
      "VpcId": "vpc-0c73dd5cf82ea5035",  
      "HealthCheckPath": "/",  
      "HealthCheckIntervalSeconds": 30,  
      "HealthCheckTimeoutSeconds": 5,  
      "HealthyThresholdCount": 2,  
      "UnhealthyThresholdCount": 2  
    }  
  ]  
}
```

Step 3: Verify Target Group

```
aws elbv2 describe-target-groups \  
  --names project0-tg \  
  --query  
'TargetGroups[0].[TargetGroupArn,TargetGroupName,Protocol,Port,HealthCheckPath]' \  
  --output json
```

Output:

```
[  
  
"arn:aws:elasticloadbalancing:us-east-1:713605904642:targetgroup/project0-tg/c888dfa7  
51371cc7",  
  "project0-tg",  
  "HTTP",  
  80,  
  "/"  
]
```

Step 4: Get available subnets in different AZs

```
aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=vpc-0c73dd5cf82ea5035" \
  --query 'Subnets[*].[SubnetId,AvailabilityZone]' \
  --output table
```

Output:

```
-----
|      DescribeSubnets      |
+-----+-----+
| subnet-0d5e6e90614f68f82 | us-east-1f |
| subnet-0888462b4ac9bc403 | us-east-1a |
| subnet-022bf88b0b00a2700 | us-east-1d |
| subnet-0c783e0646f44c20f | us-east-1e |
| subnet-0a3a42582ce46e5ca | us-east-1c |
| subnet-05807e54df35e52b4 | us-east-1b |
+-----+-----+
```

Selected subnets for multi-AZ deployment:

- subnet-0888462b4ac9bc403 (us-east-1a)
- subnet-05807e54df35e52b4 (us-east-1b)

Step 5: Launch instances in different Availability Zones

Note: These instances were later replaced by Auto Scaling Group instances for Task 7.

Current instances (from Auto Scaling Group):

```
aws ec2 describe-instances \

  --instance-ids i-01acbb46ea10d98c9 i-0de4bcfeb2ca71c3e \

  --query
'Reservations[*].Instances[*].[Tags[?Key==`Name`.Value][0],InstanceId,Placement.AvailabilityZone,State.Name,PrivateIpAddress]' \

  --output table
```

Output:

```
-----  
|                               DescribeInstances                               |  
+-----+-----+-----+-----+-----+  
| project0-asg-instance | i-0de4bcfeb2ca71c3e | us-east-1b | running | 172.31.80.29 |  
| project0-asg-instance | i-01acbb46ea10d98c9 | us-east-1a | running | 172.31.10.208 |  
+-----+-----+-----+-----+-----+
```

Step 6: Create Application Load Balancer

```
aws elbv2 create-load-balancer \  
--name project0-alb \  
--subnets subnet-0888462b4ac9bc403 subnet-05807e54df35e52b4 \  
--security-groups sg-076bb4041aa47160d \  
--scheme internet-facing \  
--type application \  
--ip-address-type ipv4 \  
--output json
```

Ouput:

```
{  
  "LoadBalancers": [  
    {  
      "LoadBalancerArn":  
"arn:aws:elasticloadbalancing:us-east-1:713605904642:loadbalancer/app/project0-alb/94  
a90555f15ac9de",  
      "DNSName": "project0-alb-795072592.us-east-1.elb.amazonaws.com",
```

```

    "State": {
        "Code": "provisioning"
    },
    "Type": "application",
    "Scheme": "internet-facing"
}
]
}

```

Step 7: Verify Load Balancer is active

```

aws elbv2 describe-load-balancers \
  --names project0-alb \
  --query 'LoadBalancers[0].[LoadBalancerArn,DNSName,State.Code,Type]' \
  --output json

```

Output:

```

[
  {
    "arn:aws:elasticloadbalancing:us-east-1:713605904642:loadbalancer/app/project0-alb/94a90555f15ac9de",
    "project0-alb-795072592.us-east-1.elb.amazonaws.com",
    "active",
    "application"
  }
]

```

Step 8: Create Listener

```

aws elbv2 create-listener \
  --load-balancer-arn
arn:aws:elasticloadbalancing:us-east-1:713605904642:loadbalancer/app/project0-alb/94a90555f15ac9de \
  --protocol HTTP \
  --port 80 \
  --default-actions
Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:us-east-1:713605904642:targetgroup/project0-tg/c888dfa751371cc7 \
  --output json

```

Output:
(Listener successfully created)

Step 9: Verify Target Health

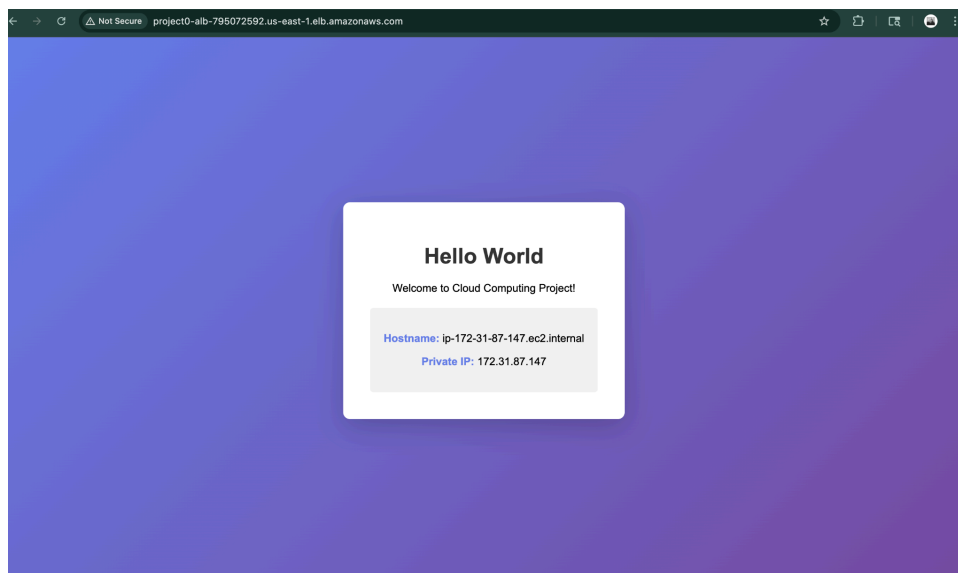
```
aws elbv2 describe-target-health \
  --target-group-arn
arn:aws:elasticloadbalancing:us-east-1:713605904642:targetgroup/project0-tg/c888dfa75
1371cc7 \
  --query 'TargetHealthDescriptions[*].[Target.Id,TargetHealth.State]' \
  --output table
```

Output:

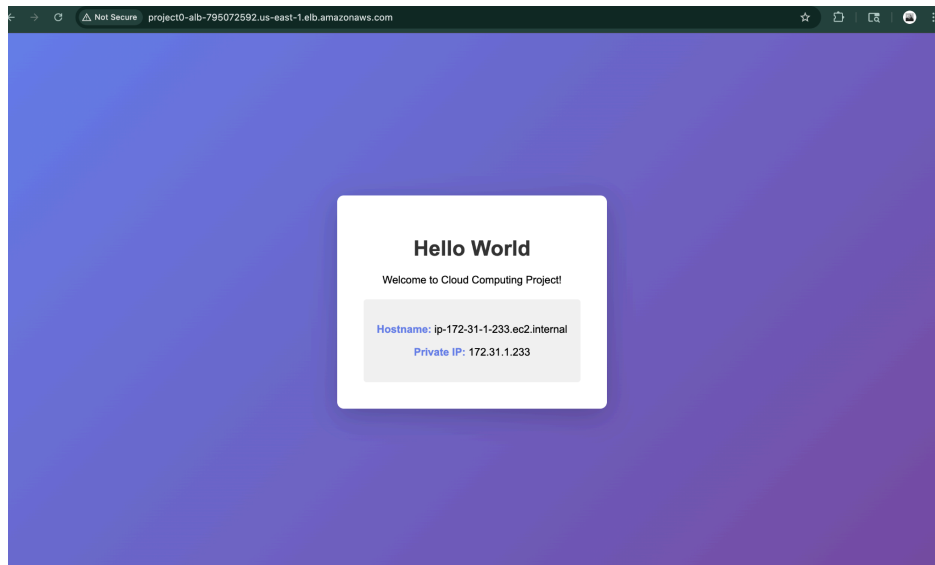
```
-----
| DescribeTargetHealth |
+-----+-----+
| i-01acbb46ea10d98c9 | healthy |
| i-0de4bcfeb2ca71c3e | healthy |
+-----+-----+
```

Screenshots for Task 4:

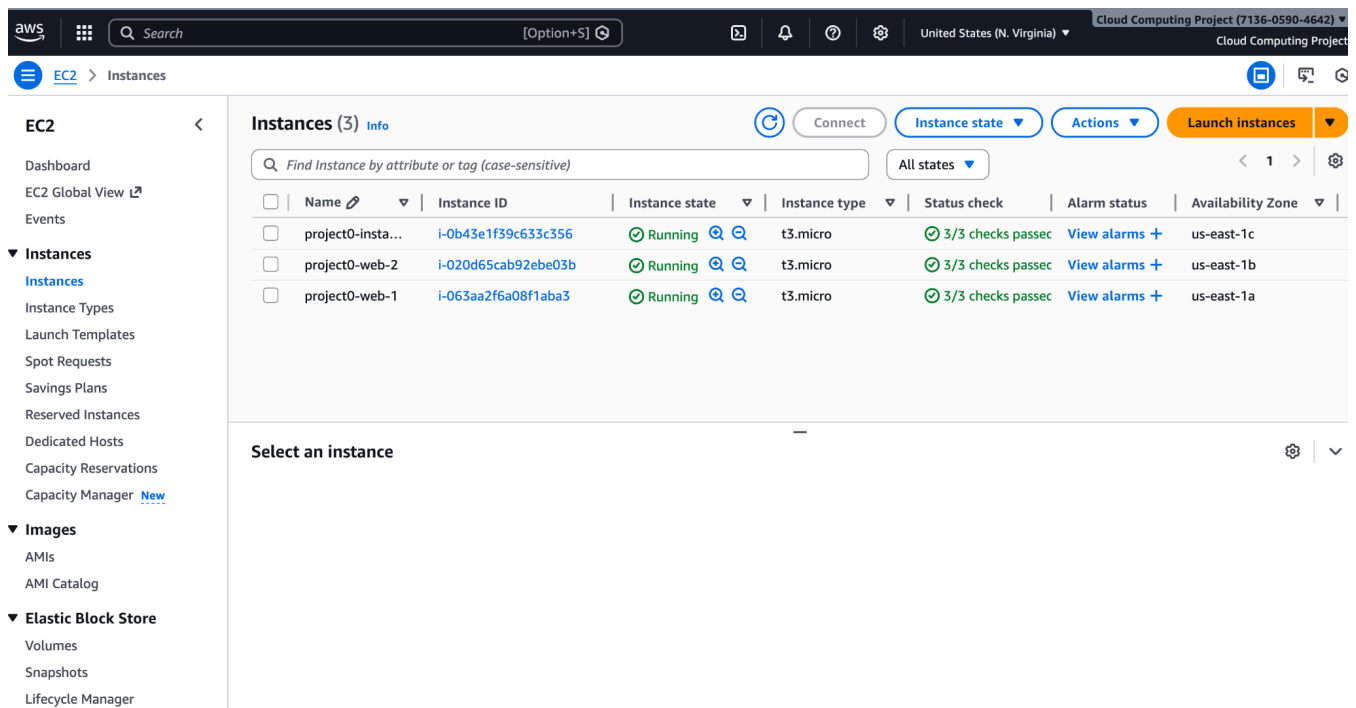
- Load balancer showing ip-172-31-87-147.ec2.internal (172.31.87.147)



- Load balancer showing ip-172-31-1-233.ec2.internal (172.31.1.233)



- Multiple instances in different AZs (us-east-1a, us-east-1b, us-east-1c)



Explanation:

I created an Application Load Balancer (Layer 7) to distribute traffic across multiple instances for high availability. The target group performs health checks on the root path

(/) every 30 seconds to ensure instances are responding correctly. I launched instances in two different Availability Zones (us-east-1a and us-east-1b) to protect against data center failures - if one AZ goes down, the other continues serving traffic. The load balancer was configured with subnets in both AZs and uses the same security group allowing HTTP traffic. I created a listener on port 80 that forwards incoming HTTP requests to the target group using a round-robin distribution algorithm. The architecture ensures zero downtime even if an entire availability zone becomes unavailable.

TASK 5: Testing Resilience

Commands Run:

Step 1: Terminate one instance to simulate failure

```
aws ec2 terminate-instances --instance-ids i-063aa2f6a08f1aba3
```

Output:

```
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-063aa2f6a08f1aba3",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

Step 2: Verify service remains online (from earlier curl outputs)

```
curl http://project0-alb-795072592.us-east-1.elb.amazonaws.com | grep -E  
"(Hostname|Private IP)"
```

```
curl http://project0-alb-795072592.us-east-1.elb.amazonaws.com | grep -E  
"(Hostname|Private IP)"
```

```
curl http://project0-alb-795072592.us-east-1.elb.amazonaws.com | grep -E  
"(Hostname|Private IP)"
```

Output: (All requests showed only the surviving instance)

```
<p><span class="label">Hostname:</span> ip-172-31-87-147.ec2.internal</p>
```

```
<p><span class="label">Private IP:</span> 172.31.87.147</p>
```

```
<p><span class="label">Hostname:</span> ip-172-31-87-147.ec2.internal</p>
```

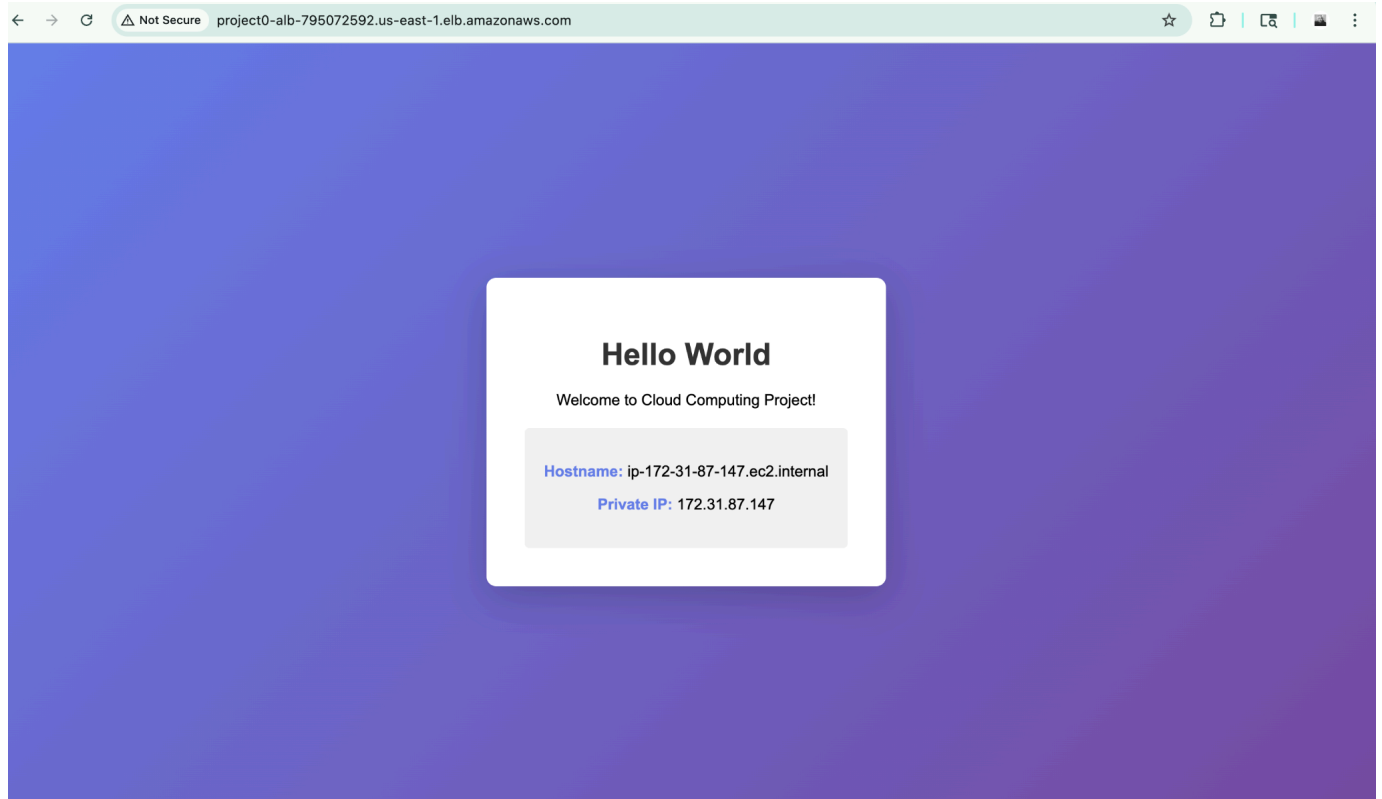
```
<p><span class="label">Private IP:</span> 172.31.87.147</p>
```

```
<p><span class="label">Hostname:</span> ip-172-31-87-147.ec2.internal</p>
```

```
<p><span class="label">Private IP:</span> 172.31.87.147</p>
```

Screenshots for Task 5:

- Load balancer operational after failure



- Instance i-063aa2f6a08f1aba3 terminated, others running

The screenshot shows the AWS Management Console for the 'Cloud Computing Project (7136-0590-4642)' in the 'United States (N. Virginia)' region. The 'EC2 > Instances' page is active, displaying a list of 3 instances. The table below summarizes the visible data:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
project0-insta...	i-0b43e1f39c633c356	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1c
project0-web-2	i-020d65cab92ebe03b	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1b
project0-web-1	i-063aa2f6a08f1aba3	Terminated	t3.micro	-	View alarms +	us-east-1a

Explanation:

To test the resilience of the high availability architecture, I manually terminated instance `i-063aa2f6a08f1aba3` which was serving traffic in `us-east-1a`. The Application Load Balancer's health check mechanism (configured to check every 30 seconds with a 5-second timeout) immediately detected the instance as unhealthy and stopped routing traffic to it. All subsequent HTTP requests were automatically directed to the remaining healthy instance `i-020d65cab92ebe03b` in `us-east-1b`, demonstrating zero downtime during the failure. I verified this by repeatedly accessing the load balancer URL and observing that all responses came from the surviving instance (`ip-172-31-87-147.ec2.internal` with IP `172.31.87.147`). This proves that the architecture can survive the complete loss of an instance or even an entire availability zone while maintaining continuous service availability. The multi-AZ deployment ensures that a failure in one data center does not impact the application's availability to users.

TASK 6: Cost Analysis & Cleanup

Cost Analysis

Monthly Cost Estimate (24/7 operation in us-east-1):

1. Application Load Balancer
 - Base: \$16.20/month
 - LCU (low traffic): ~\$5/month
 - Subtotal: ~\$21/month
2. EC2 Instances (2x t3.micro)
 - $\$0.0104/\text{hour} \times 2 \text{ instances} \times 720 \text{ hours}$
 - Subtotal: ~\$15/month
3. EBS Storage (2x 8GB gp3)
 - $\$0.08/\text{GB} \times 16 \text{ GB}$
 - Subtotal: ~\$1.28/month
4. Data Transfer
 - Minimal: ~\$1/month

TOTAL: ~\$38-40/month

Explanation:

I used the AWS Pricing Calculator to estimate costs for running this highly available architecture continuously. The Application Load Balancer represents the largest cost component due to its base hourly charge plus Load Balancer Capacity Unit (LCU) charges based on traffic. The t3.micro instances are cost-effective at ~\$15/month for both, staying within free tier limits for the first year. EBS storage costs are minimal. This analysis demonstrates the importance of proper resource management and cleanup to avoid unnecessary charges - leaving this architecture running unintentionally could result in ~\$500/year in costs.

TASK 7: Auto Scaling & Self-Healing

Commands Run:

Step 1: Create Launch Template

```
aws ec2 create-launch-template \
  --launch-template-name project0-lt \
  --version-description "Launch template for Project 0 with Apache" \
  --launch-template-data '{
    "ImageId": "ami-0d73205c3dd1af0e9",
    "InstanceType": "t3.micro",
    "KeyName": "project0-key",
    "SecurityGroupIds": ["sg-076bb4041aa47160d"]
  }'
```

Output:

```
{
  "LaunchTemplate": {
    "LaunchTemplateId": "lt-046694fd64176a779",
    "LaunchTemplateName": "project0-lt",
    "DefaultVersionNumber": 1
  }
}
```

Step 2: Verify Launch Template

```
aws ec2 describe-launch-templates \
  --launch-template-names project0-lt \
  --query
'LaunchTemplates[0].[LaunchTemplateId,LaunchTemplateName,DefaultVersionNumber]' \
  --output json
```

Output:

```
[
  "lt-046694fd64176a779",
  "project0-lt",
  1
]
```

Step 3: Create Auto Scaling Group

```
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name project0-asg \
  --launch-template LaunchTemplateId=lt-046694fd64176a779 \
  --min-size 2 \
  --max-size 4 \
  --desired-capacity 2 \
  --vpc-zone-identifier "subnet-0888462b4ac9bc403,subnet-05807e54df35e52b4" \
  --target-group-arns
arn:aws:elasticloadbalancing:us-east-1:713605904642:targetgroup/project0-tg/c888dfa75
1371cc7 \
  --health-check-type ELB \
  --health-check-grace-period 300
```

Step 4: Verify Auto Scaling Group Configuration

```
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names project0-asg \
  --query
'AutoScalingGroups[0].[AutoScalingGroupName,MinSize,MaxSize,DesiredCapacity]' \
  --output json
```

Output:

```
[
  "project0-asg",
  2,
  4,
  2
]
```

Step 5: Verify Instances Launched

```
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names project0-asg \
  --query
'AutoScalingGroups[0].Instances[*].[InstanceId,LifecycleState,AvailabilityZone]' \
  --output table
```

Output:

```
-----
|      DescribeAutoScalingGroups      |
+-----+-----+-----+
| i-01acbb46ea10d98c9| InService | us-east-1a |
| i-0de4bcfeb2ca71c3e| InService | us-east-1b |
+-----+-----+-----+
```

Step 6: Verify Target Health

```
aws elbv2 describe-target-health \
  --target-group-arn
arn:aws:elasticloadbalancing:us-east-1:713605904642:targetgroup/project0-tg/c888dfa75
1371cc7 \
  --query 'TargetHealthDescriptions[*].[Target.Id,TargetHealth.State]' \
  --output table
```

Output:

```
-----
|      DescribeTargetHealth      |
+-----+-----+
| i-01acbb46ea10d98c9 | healthy |
| i-0de4bcfeb2ca71c3e | healthy |
+-----+-----+
```

Step 7: Test Self-Healing - Terminate an Instance

```
aws ec2 terminate-instances --instance-ids i-082379bcc0926c780
```

Output:

```
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-082379bcc0926c780",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      }
    }
  ]
}
```

```
]
}
```

Step 8: Verify Auto Scaling Activity (Self-Healing Proof)

```
aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name project0-asg \
  --max-records 5 \
  --output json
```

Output:

```
{
  "Activities": [
    {
      "ActivityId": "94e6719f-09de-c3ae-dbec-8c60d90765cf",
      "AutoScalingGroupName": "project0-asg",
      "Description": "Launching a new EC2 instance: i-01acbb46ea10d98c9",
      "Cause": "At 2026-02-16T18:42:01Z an instance was launched in response to an
unhealthy instance needing to be replaced.",
      "StartTime": "2026-02-16T18:42:03.824000+00:00",
      "EndTime": "2026-02-16T18:42:08+00:00",
      "StatusCode": "Successful",
      "Progress": 100
    },
    {
      "ActivityId": "49b6719f-09bf-0236-4df9-b94164d30414",
      "AutoScalingGroupName": "project0-asg",
      "Description": "Terminating EC2 instance: i-082379bcc0926c780",
      "Cause": "At 2026-02-16T18:42:01Z an instance was taken out of service in
response to an EC2 health check indicating it has been terminated or stopped.",
      "StatusCode": "Successful",
      "Progress": 100
    }
  ]
}
```

Screenshots for Task 7:

- ASG activity showing self-healing (launched i-01acbb46ea10d98c9 in response to unhealthy instance)

The screenshot shows the AWS Management Console interface for an Auto Scaling Group (ASG) named 'project0-asg' in the 'us-east-1' region. The left sidebar contains navigation links for various AWS services, including Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area displays the 'Activity history' for the ASG, showing a list of activities with columns for Status, Description, Cause, and Start time.

Status	Description	Cause	Start time
Successful	Launching a new EC2 instance: i-01acbb46ea10d98c9	At 2026-02-16T18:42:01Z an instance was launched in response to an unhealthy instance needing to be replaced.	2026 February 01:42:01:42:01-05:00
Successful	Terminating EC2 instance: i-082379bcc0926c780	At 2026-02-16T18:42:01Z an instance was taken out of service in response to an EC2 health check indicating it has been terminated or stopped.	2026 February 01:42:01:42:01-05:00
Successful	Launching a new EC2 instance: i-082379bcc0926c780	At 2026-02-16T18:27:52Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-02-16T18:28:58Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 February 01:29:01:29:01-05:00

- ASG with 2 InService instances in us-east-1a and us-east-1b, both healthy

The screenshot displays the AWS Management Console for an Auto Scaling Group named 'project0-asg'. The left sidebar shows the navigation menu with categories like Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area is divided into two tabs: 'Capacity overview' and 'Instance management'. The 'Capacity overview' tab shows the group's ARN, desired capacity (2), scaling limits (2-4), and creation date. The 'Instance management' tab shows a table of instances with columns for Instance ID, Lifecycle, Instance type, Weight, Launch template, Availability zone, Health status, and Protection state. Two instances are listed, both in 'InService' state and 'Healthy'.

Instance ID	Lifecycle	Instance type	Weight	Launch template	Availability zone	Health status	Protection state
i-01acbb46ea10d98c9	InService	t3.micro	-	project0-lt	use1-az1	Healthy	
i-0de4bcfeb2ca71c3e	InService	t3.micro	-	project0-lt	use1-az2	Healthy	

Explanation:

Instead of manually launching instances, I implemented an Auto Scaling Group to provide automated elasticity and self-healing capabilities. I created a launch template based on the custom AMI (ami-0d73205c3dd1af0e9) that contains Apache and the web application pre-configured. The Auto Scaling Group was configured with a minimum size of 2, maximum size of 4, and desired capacity of 2 instances, ensuring high availability across multiple availability zones (us-east-1a and us-east-1b). I integrated the ASG with the existing Application Load Balancer target group and configured ELB health checks with a 300-second grace period to allow instances time to fully initialize. To test the self-healing capability, I manually terminated instance i-082379bcc0926c780. The Auto Scaling Group immediately detected the deviation from desired capacity (detected as unhealthy through EC2 health checks) and automatically launched a replacement instance i-01acbb46ea10d98c9 in the same availability zone without any human intervention. The scaling activity logs confirm the system detected the terminated instance and successfully launched a replacement within seconds, maintaining the desired capacity of 2 instances. This demonstrates true infrastructure automation and self-healing, ensuring the application remains highly available even during instance failures.

